

Mediapipe Human Pose Tracking (No-PTZ)

1. Experiment Objective

MediaPipe Pose is used to detect 33 keypoints of the human body, and the car rotates to follow so that the target person stays at the center of the frame. It supports Tracking mode (rotation only) and Follow mode (rotation + distance keeping).

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
MediaPipe Pose	A lightweight human pose estimation model provided by Google that can detect the coordinates of 33 human keypoints in real time
33 Pose Keypoints	The human skeleton nodes defined by MediaPipe, including nose, eyes, shoulders, elbows, wrists, hips, knees, ankles, etc.
Body Center of Gravity Calculation	Combines keypoints such as the hip center and shoulder center to compute the reference position of the human body in the frame
Pose Skeleton Connections	Connects the 33 keypoints according to the human topology to form a skeleton
PD Control	Proportional-derivative controller that computes the body angular velocity based on the body center deviation and its rate of change
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
  → pose_tracker_node
    └─ Image processing (15Hz): image conversion → flip → MediaPipe
        Pose.process() → 33 keypoints → center of gravity calculation
    └─ Body control (10Hz): pixel error → PD angular velocity + bounding box
        width distance following → /cmd_vel (BEST_EFFORT)
```

Requires a MediaPipe Python environment (ROS 2 and workspace environment).

3. Core Topic Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.30~0.30 m/s	$\neq$ forward, $=$ backward
			angular.z	-0.60~0.60 rad/s	$\neq$ turn left, $=$ turn right

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted)

2. Start the Agent + Camera + Joint Model (Terminal 1)

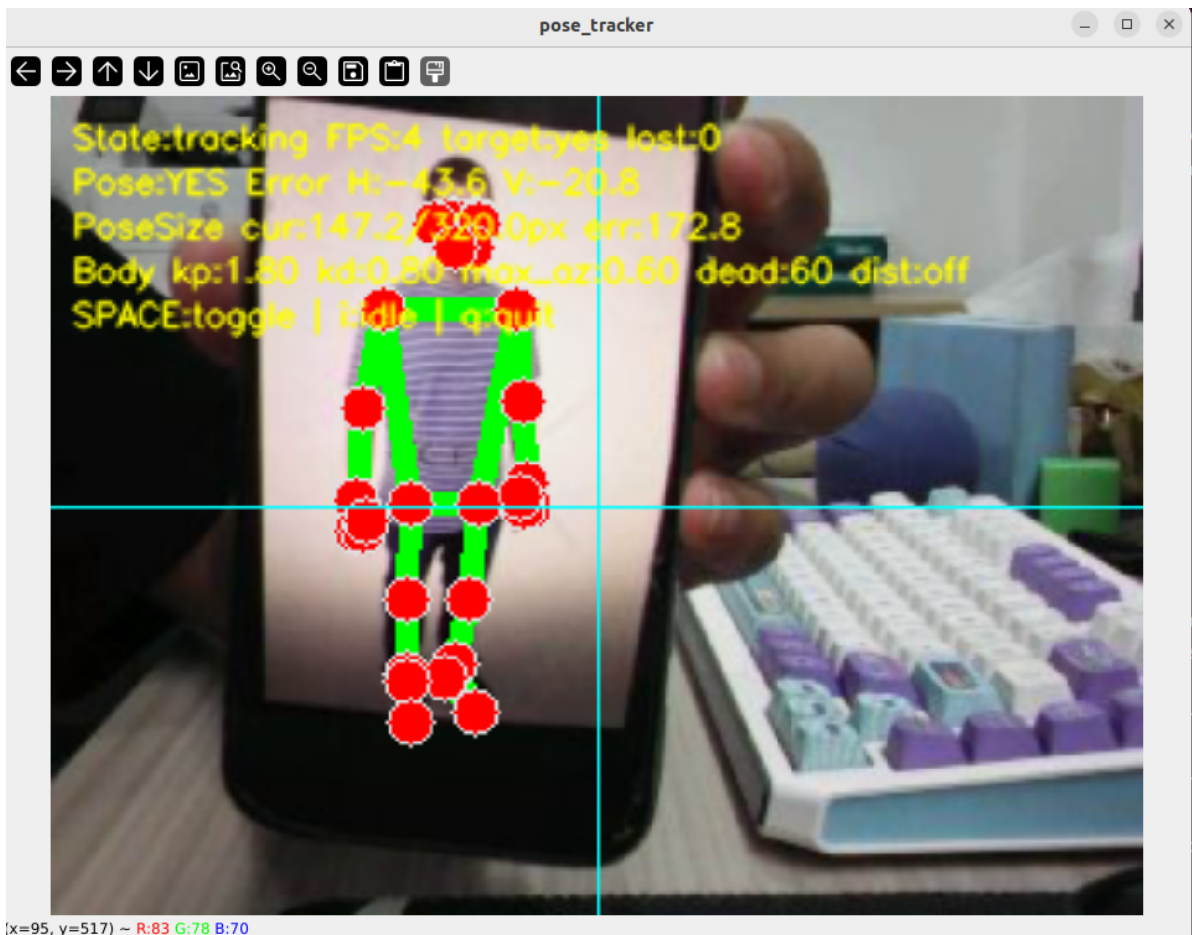
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom: $ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | Init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment can1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Human Pose Tracking (Terminal 2)

Tracking mode:

```
ros2 launch microros_v2_no_ptz_visual_control_pkg pose_tracker.launch.py
```



6. Operation Instructions

Key	Function
Space	Toggle tracking/standby state
i / I	Switch to the standby (IDLE) state
q / Q / ESC	Exit the program

4. Experiment Phenomenon

- When the person moves left or right, the car automatically rotates to track and keep them at the center of the frame
- The 33 human keypoints and skeleton connections are displayed in real time
- Follow mode: the car backs away when the person moves closer and moves forward when they move away
- With multiple people, the tracking target may switch

5. Code Explanation

Source path:

- Launch (track):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/pose_tracker.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/pose_tracker_node.py`

- Pose detector:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/pose_detector.py
```

1. Core Node Analysis

`pose_tracker_node` uses `PoseDetector` to wrap the MediaPipe Pose model. Core workflow:

- **Image processing** (15Hz): raw Image frame conversion → flip → `PoseDetector.findPose()` → extract 33 keypoints
- **Center of gravity calculation**: takes keypoints such as the hip center (midpoint of the left and right hip joints) plus the shoulder center to comprehensively compute the human reference position
- **Body control** (10Hz): deviation between the center x-coordinate and the image center → PD computes angular velocity; bounding box width → PD computes linear velocity

2. Key Parameters

Parameter definition file:

- Launch:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/pose_tracker.launch.py
```

- Node:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/pose_tracker_node.py
```

Parameter	Type	Default	Description
<code>pose_detection_confidence</code>	float	0.5	Pose detection confidence threshold
<code>pose_tracking_confidence</code>	float	0.5	Pose tracking confidence threshold
<code>deadband_x_pixel</code>	float	60.0 px	Horizontal deadband
<code>angular_kp</code>	float	1.8	Body angle PD proportional coefficient
<code>angular_kd</code>	float	0.8	Body angle PD derivative coefficient
<code>max_angular_z</code>	float	0.6 rad/s	Maximum body steering speed
<code>enable_distance_follow</code>	bool	false	Whether to enable distance following
<code>target_pose_size_px</code>	float	320.0 px	Target human bounding box width for distance following
<code>distance_tolerance_pose_size_px</code>	float	10.0 px	Distance-following deadband
<code>distance_kp</code>	float	0.02	Distance PD proportional coefficient
<code>max_linear_x</code>	float	0.30 m/s	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>pose_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	MediaPipe human pose tracking with 33 keypoints + PD body control

6. Common Problems

Problem	Cause	Solution
Tracking lag	Pose detection is computationally heavy and the frame rate is low	Lower <code>image_processing_rate_hz</code> or reduce the processing resolution
Person is not detected	Distance too far or part of the body is out of the frame	Keep the whole body in the frame and move closer to the camera
Multiple-person interference	The tracking target switches when several people are in the frame	Ensure only the target person is in the frame